

Why this matters for cybersecurity — and for SageMath users

Here is where things get especially relevant to our discussion on cybersecurity using SageMath. Quantum computing presents a serious threat to many existing public-key cryptographic systems, especially those based on RSA and ECC. These systems rely on the computational hardness of problems like integer factorization and discrete logarithms — problems that quantum algorithms like Shor's algorithm can solve exponentially faster than classical algorithms. This means that once practical quantum computers become available, much of today's encryption could be broken in a matter of seconds — making sensitive data, secure communication, and digital authentication systems vulnerable to quantum attacks.

In this article, we will begin exploring cryptanalysis. With quantum computing on the horizon, cryptanalysis — the science of breaking codes — could become orders of magnitude more powerful than anything currently possible, outpacing even the most advanced supercomputers. So, as we dive into these cryptographic algorithms and their SageMath implementations, it is essential to keep one eye on the future — a future that might arrive much sooner than expected.

Cryptanalysis

Before we delve into cryptanalysis, it is important to first understand the concept of a brute-force attack — a method where every possible key is systematically tried until the correct one is found. On classical computers (the computers we use now), brute-force attacks become practically infeasible as the key size increases, thanks to the exponential growth in the number of possible keys. However, quantum computers can perform certain types of computations much faster than classical ones. This raises the concern that quantum computing systems could significantly accelerate brute-force attacks, reducing the time required to break encryption that is currently considered secure.

Now, let us examine how quantum computing impacts brute-force attacks and cryptanalysis. One of the key quantum algorithms relevant here is Grover's algorithm, which provides a significant speedup for brute-force search problems. While classical algorithms require $O(N)$ time to search an unstructured space of size N , Grover's algorithm can accomplish the same task in $O(\sqrt{N})$ time. This means that a 128-bit key, which currently offers strong security, would provide only 64-bit effective security against an adversary with access to a quantum computer. In other words, quantum computers could weaken symmetric-key encryption, even if they don't break it entirely. It is important to understand that the main threat quantum computing poses to symmetric-key encryption lies in its general computational speedup, rather than in any specific vulnerability in the encryption algorithms

themselves. With that understanding, let us now explore a few examples of cryptanalysis applied to some widely used symmetric-key encryption schemes.

In the field of cryptanalysis, several techniques are used to evaluate and potentially break encryption schemes. These include ciphertext-only attacks, where the attacker has access only to encrypted messages; known-plaintext attacks, where some pairs of plaintext and ciphertext are known; chosen-plaintext attacks, where the attacker can encrypt messages of their choosing; and chosen-ciphertext attacks, where the attacker can decrypt selected ciphertexts without knowing the key.

In a known-plaintext attack (KPA), the attacker uses known pairs of plaintext and corresponding ciphertext to infer the key or decrypt other ciphertexts. This approach is more powerful than ciphertext-only attacks and was used successfully against classical ciphers like the Vigenère cipher. A chosen-plaintext attack (CPA) is even stronger — here, the attacker can choose arbitrary plaintexts and observe their corresponding ciphertexts. This helps reveal patterns in the encryption process and is commonly used to evaluate modern block and stream ciphers. In a chosen-ciphertext attack (CCA), the attacker can select ciphertexts and obtain their decrypted plaintexts. All these attack models—ciphertext-only, known-plaintext, chosen-plaintext, and chosen-ciphertext—will become significantly more potent with the advent of quantum computers.

Now, consider the SageMath program named 'kpa28.sage' shown below (line numbers are included for clarity), which implements a known-plaintext attack (KPA) on the Vigenère cipher. Recall that we discussed the Vigenère cipher in detail in the seventh article in this series (published in the November 2024 issue of *OSFY*).

```
1. alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
2. letter_to_index = {c: i for i, c in enumerate(alphabet)}
3. index_to_letter = {i: c for i, c in enumerate(alphabet)}
4. def vigenere_encrypt(plaintext, key):
5.     ciphertext = ""
6.     for i, c in enumerate(plaintext):
7.         pi = letter_to_index[c]
8.         ki = letter_to_index[key[i % len(key)]]
9.         ci = (pi + ki) % 26
10.        ciphertext += index_to_letter[ci]
11.    return ciphertext
12. plaintext = "HELLOOPENSOURCEFORYOU"
13. key = "DEEPU"
14. ciphertext = vigenere_encrypt(plaintext, key)
15. print("Ciphertext:", ciphertext)
16. recovered_key = ""
17. key_length = len(key)
18. for i in range(key_length):
19.     pi = letter_to_index[plaintext[i]]
```